

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Логика и основы алгоритмизации в инженерных задачах»
на тему: «Динамические списки»

Выполнили ст.гр. 25ВВВ3:

Волков И.Р.

Итянина В.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н, ст. преподаватель, Деев М.В.

Пенза 2026

Цель работы: изучить динамические структуры данных на языке С и реализовать на основе односвязного списка приоритетную очередь, очередь и стек.

Ход работы

Задание 1: реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).

Код:

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <locale.h>

struct node
{
    char inf[256];
    int priority;
    struct node* next;
};

struct node* head = NULL;
struct node* last = NULL;

struct node* get_struct(void)
{
    struct node* p = NULL;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL)
    {
        printf("Ошибка выделения памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%255s", p->inf);

    printf("Введите приоритет объекта: ");
    scanf("%d", &p->priority);

    p->next = NULL;

    return p;
}
```

```

void spstore(void)
{
    struct node* p = get_struct();
    struct node* current;

    if (head == NULL)
    {
        head = p;
        last = p;
        return;
    }

    if (p->priority > head->priority)
    {
        p->next = head;
        head = p;
        return;
    }

    current = head;

    while (current->next != NULL &&
           current->next->priority >= p->priority)
    {
        current = current->next;
    }

    p->next = current->next;
    current->next = p;

    if (p->next == NULL)
    {
        last = p;
    }
}

void review(void)
{
    struct node* current = head;

    if (head == NULL)
    {
        printf("Список пуст\n");
        return;
    }

    printf("\nСодержимое списка:\n");

    while (current != NULL)
    {
        printf("Название: %s | Приоритет: %d\n",

```

```

        current->inf,
        current->priority);

        current = current->next;
    }
}

void del_first(void)
{
    struct node* temp;

    if (head == NULL)
    {
        printf("Список пуст\n");
        return;
    }

    temp = head;
    head = head->next;

    free(temp);

    if (head == NULL)
    {
        last = NULL;
    }

    printf("Первый элемент удалён\n");
}

void clear_list(void)
{
    struct node* temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }

    last = NULL;
}

int main(void)
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    int choice;

    do

```

```

{
    printf("\n Приоритетная очередь \n");
    printf("1. Добавить элемент\n");
    printf("2. Просмотреть список\n");
    printf("3. Удалить первый элемент\n");
    printf("0. Выход\n");
    printf("Ваш выбор: ");

    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            spstore();
            break;

        case 2:
            review();
            break;

        case 3:
            del_first();
            break;

        case 0:
            clear_list();
            printf("Выход из программы\n");
            break;

        default:
            printf("Неверный выбор\n");
    }

    } while (choice != 0);

    return 0;
}

```

Результат работы программы представлен на рисунке 1.

```

Приоритетная очередь
1. Добавить элемент
2. Просмотреть список
3. Удалить первый элемент
0. Выход
Ваш выбор: 1
Введите название объекта: кошка
Введите приоритет объекта: 1

Приоритетная очередь
1. Добавить элемент
2. Просмотреть список
3. Удалить первый элемент
0. Выход
Ваш выбор: 1
Введите название объекта: собака
Введите приоритет объекта: 3

Приоритетная очередь
1. Добавить элемент
2. Просмотреть список
3. Удалить первый элемент
0. Выход
Ваш выбор: 1
Введите название объекта: змея
Введите приоритет объекта: 2

Приоритетная очередь
1. Добавить элемент
2. Просмотреть список
3. Удалить первый элемент
0. Выход
Ваш выбор: 1
Введите название объекта: корова
Введите приоритет объекта: 5

Приоритетная очередь
1. Добавить элемент
2. Просмотреть список
3. Удалить первый элемент
0. Выход
Ваш выбор: 2

Содержимое списка:
Название: корова | Приоритет: 5
Название: собака | Приоритет: 3
Название: змея | Приоритет: 2
Название: кошка | Приоритет: 1

Приоритетная очередь
1. Добавить элемент
2. Просмотреть список
3. Удалить первый элемент
0. Выход
Ваш выбор: 3
Первый элемент удалён

Приоритетная очередь
1. Добавить элемент
2. Просмотреть список
3. Удалить первый элемент
0. Выход
Ваш выбор: 2

Содержимое списка:
Название: собака | Приоритет: 3
Название: змея | Приоритет: 2
Название: кошка | Приоритет: 1

Приоритетная очередь
1. Добавить элемент
2. Просмотреть список
3. Удалить первый элемент
0. Выход
Ваш выбор:

```

Рис.1 – результат работы программы к заданию 1.

Задание 2: на основе приведенного кода реализуйте структуру данных Очередь.

Код:

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <locale.h>

struct node
{
    char inf[256];
    struct node* next;
};

struct node* head = NULL;
struct node* last = NULL;

struct node* get_struct(void)
{
    struct node* p;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL)
    {
        printf("Ошибка выделения памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%255s", p->inf);

    p->next = NULL;

    return p;
}

void enqueue(void)
{
    struct node* p = get_struct();

    if (head == NULL)
    {
        head = p;
        last = p;
    }
    else
```

```

        {
            last->next = p;
            last = p;
        }

        printf("Элемент добавлен в очередь\n");
    }

void dequeue(void)
{
    struct node* temp;

    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    temp = head;
    head = head->next;

    printf("Удалённый элемент: %s\n", temp->inf);

    free(temp);

    if (head == NULL)
    {
        last = NULL;
    }
}

void review(void)
{
    struct node* current = head;

    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    while (current != NULL)
    {
        printf("%s\n", current->inf);
        current = current->next;
    }
}

void clear_list(void)
{
    struct node* temp;

    while (head != NULL)

```

```

    {
        temp = head;
        head = head->next;
        free(temp);
    }

    last = NULL;
}

int main(void)
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    int choice;

    do
    {
        printf("\n Очередь \n");
        printf("1. Добавить элемент\n");
        printf("2. Удалить элемент\n");
        printf("3. Просмотреть очередь\n");
        printf("0. Выход\n");
        printf("Ваш выбор: ");

        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                enqueue();
                break;

            case 2:
                dequeue();
                break;

            case 3:
                review();
                break;

            case 0:
                clear_list();
                printf("Выход из программы\n");
                break;

            default:
                printf("Неверный выбор\n");
        }
    } while (choice != 0);
}

```

```
    return 0;  
}
```

Результат работы программы представлен на рисунке 2.

```
Очередь  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть очередь  
0. Выход  
Ваш выбор: 1  
Введите название объекта: маша  
Элемент добавлен в очередь  
  
Очередь  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть очередь  
0. Выход  
Ваш выбор: 1  
Введите название объекта: стеша  
Элемент добавлен в очередь  
  
Очередь  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть очередь  
0. Выход  
Ваш выбор: 1  
Введите название объекта: аглая  
Элемент добавлен в очередь  
  
Очередь  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть очередь  
0. Выход  
Ваш выбор: 1  
Введите название объекта: николай  
Элемент добавлен в очередь  
  
Очередь  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть очередь  
0. Выход  
Ваш выбор: 3  
маша  
стеша  
аглая  
николай  
  
Очередь  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть очередь  
0. Выход  
Ваш выбор: 2  
Удалённый элемент: маша  
  
Очередь  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть очередь  
0. Выход  
Ваш выбор: 3  
стеша  
аглая  
николай  
  
Очередь  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть очередь  
0. Выход  
Ваш выбор:
```

Рис.2 – результат работы программы к заданию 2.

Задание 3: на основе приведенного кода реализуйте структуру данных Стек.

Код:

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <locale.h>

struct node
{
    char inf[256];
    struct node* next;
};

struct node* head = NULL;

struct node* get_struct(void)
{
    struct node* p;

    p = (struct node*)malloc(sizeof(struct node));

    if (p == NULL)
    {
        printf("Ошибка выделения памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%255s", p->inf);

    p->next = NULL;

    return p;
}

void push(void)
{
    struct node* p = get_struct();

    p->next = head;
    head = p;

    printf("Элемент добавлен в стек\n");
}

void pop(void)
{
    struct node* temp;
```

```

    if (head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    temp = head;
    head = head->next;

    printf("Удалённый элемент: %s\n", temp->inf);

    free(temp);
}

void review(void)
{
    struct node* current = head;

    if (head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    while (current != NULL)
    {
        printf("%s\n", current->inf);
        current = current->next;
    }
}

void clear_list(void)
{
    struct node* temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main(void)
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    int choice;

    do
    {

```

```

printf("\n Стек \n");
printf("1. Добавить элемент\n");
printf("2. Удалить элемент\n");
printf("3. Просмотреть стек\n");
printf("0. Выход\n");
printf("Ваш выбор: ");

scanf("%d", &choice);

switch (choice)
{
case 1:
    push();
    break;

case 2:
    pop();
    break;

case 3:
    review();
    break;

case 0:
    clear_list();
    printf("Выход из программы\n");
    break;

default:
    printf("Неверный выбор\n");
}

} while (choice != 0);

return 0;
}

```

Результат работы программы представлен на рисунке 3.

```

Стек
1. Добавить элемент
2. Удалить элемент
3. Просмотреть стек
0. Выход
Ваш выбор: 1
Введите название объекта: маша
Элемент добавлен в стек

Стек
1. Добавить элемент
2. Удалить элемент
3. Просмотреть стек
0. Выход
Ваш выбор: 1
Введите название объекта: даша
Элемент добавлен в стек

Стек
1. Добавить элемент
2. Удалить элемент
3. Просмотреть стек
0. Выход
Ваш выбор: 1
Введите название объекта: вана
Элемент добавлен в стек

Стек
1. Добавить элемент
2. Удалить элемент
3. Просмотреть стек
0. Выход
Ваш выбор: 1
Введите название объекта: вика
Элемент добавлен в стек

Стек
1. Добавить элемент
2. Удалить элемент
3. Просмотреть стек
0. Выход
Ваш выбор: 3
вика
вана
даша
маша

Стек
1. Добавить элемент
2. Удалить элемент
3. Просмотреть стек
0. Выход
Ваш выбор: 2
Удалённый элемент: вика

Стек
1. Добавить элемент
2. Удалить элемент
3. Просмотреть стек
0. Выход
Ваш выбор: 3
вана
даша
маша

Стек
1. Добавить элемент
2. Удалить элемент
3. Просмотреть стек
0. Выход
Ваш выбор:

```

Рис.3 – результат работы программы к заданию 3.

Вывод: в ходе работы были реализованы три структуры данных на базе односвязного списка: приоритетная очередь (сортировка по убыванию приоритета), очередь и стек. Закреплены навыки работы с указателями, динамической памятью (malloc, free) и операциями вставки и удаления элементов.